



**Problem Solving With C**  
**UE24CS151B**

By,  
**Prof. Bivas Bhattacharya,**  
**Assistant Professor**

**Question 1:**

- **Question:** Explain the purpose of the `printf()` function in C, including its syntax and return value. How does the format string control the output?
- **Answer:**
- `printf()` is the formatted output function in C, used to display data to the standard output (`stdout`), which is usually the console.
- **Syntax:** `int printf(const char *format, arg1, arg2, ...)` where `format` is a string literal that controls the output formatting and `arg1`, `arg2`, etc. are the variables/expressions to be printed. The `const char *format` parameter is a pointer to a constant character string, indicating that the format string should not be modified within the function. The ellipsis (...) indicates that `printf` can accept a variable number of arguments.
- **Return Value:** On success, `printf()` returns the number of characters successfully written to the output. On failure, it returns a negative value.
- **Format String Control:** The format string contains two types of items: ordinary characters (printed directly) and conversion specifications (beginning with %). Conversion specifications determine how the subsequent arguments are formatted and displayed. For example, `%d` is used for integers, `%f` for floating-point numbers, and `%s` for strings. The format string can also include flags, field widths, and precision specifiers to further refine the output's appearance.

**Question 2:**

- **Question:** Describe the components of a `printf()` format string, including `flags`, `field_width`, `precision`, and `conversion_character`. Give examples of how each component can be used to modify the output.
- **Answer:**
- The format string in `printf()` has the form `%[flags][field_width][.precision]conversion_character`. The minimum requirement is % and a conversion character (e.g., `%d`).

- **Flags (optional):** Modify the output format (e.g., justification, sign, prefixes). Examples: `-` (left-justify), `+` (always show sign), `0` (pad with zeros).
- **field\_width (optional):** Specifies the minimum number of characters to be printed. If the output is shorter, it's padded with spaces (or zeros if the `0` flag is used). Example: `%10d` (prints an integer, using at least 10 characters).
- **.precision (optional):** Specifies the maximum number of characters to print (for strings) or the number of digits after the decimal point (for floating-point numbers). Example: `%.2f` (prints a float with 2 decimal places), `%.10s` (prints at most the first 10 characters of a string).
- **conversion\_character:** Specifies the data type of the argument. Examples: `d` (decimal integer), `x` (hexadecimal integer), `o` (octal integer), `f` (float), `c` (character), `s` (string), `p` (pointer), `lf` (double).

### Question 3:

- **Question:** Explain the purpose of escape sequences in C, and provide five examples with a brief explanation of what each does.
- **Answer:**
- Escape sequences are special character combinations used in strings to represent characters that are difficult or impossible to type directly (e.g., newline, tab, backslash, quotes) or to control output formatting. They start with a backslash (`\`).
- **Examples:**
- `\n`: Newline. Moves the cursor to the beginning of the next line.
- `\t`: Horizontal Tab. Inserts horizontal whitespace, moving the cursor to the next tab stop.
- `\\`: Backslash. Inserts a literal backslash character.
- `\"`: Double Quote. Inserts a double quote character within a string literal.
- `\a`: Alarm/Beep. Generates a bell sound (may not work on all systems).

### Question 4:

- **Question:** Describe the purpose of the `scanf()` function in C, including its syntax, and the importance of the address operator (`&`) when using it. Explain the possible return values of `scanf()` and what they indicate.
- **Answer:**
- `scanf()` is the formatted input function in C, used to read data from the standard input (`stdin`), which is usually the keyboard. It parses the input according to a specified format and stores the results in variables.
- **Syntax:** `int scanf(const char *format, ...)` where `format` is a string literal that specifies the expected input format, and `...` represents one or more pointers to variables where the input values will be stored.
- **Importance of & (Address Operator):** `scanf()` requires the `addresses` of the variables where it should store the input values. The address operator (`&`) is used to obtain the memory address of a variable.

Without `&`, `scanf()` would attempt to write to the variable's *value* rather than its location in memory, leading to undefined behavior (potentially crashing the program).

- **Return Values:**
- `> 0`: The number of input items successfully converted and assigned.
- `0`: No input items were assigned (e.g., input didn't match the format string).
- `< 0`: A read error was encountered, or the end-of-file (EOF) was reached before any items could be assigned. A common negative value is `-1`.

#### Question 5:

- **Question:** The slides include the statement `sizeof(short int) <= sizeof(int) <= sizeof(float) <= sizeof(double)`. Explain what the `sizeof` operator does, and what this statement implies about the relative sizes (in bytes) of these data types in C. Why are these sizes implementation-dependent?
- **Answer:**
- The `sizeof` operator returns the size (in bytes) of a data type or variable. It's a compile-time operator, meaning the size is determined at compilation.
- The statement `sizeof(short int) <= sizeof(int) <= sizeof(float) <= sizeof(double)` means that the size of `short int` is less than or equal to the size of `int`, which is less than or equal to the size of `float`, which is less than or equal to the size of `double`. It provides a *minimum* relative size order.
- **Implementation-Dependent:** The actual sizes of these data types (in bytes) can vary depending on the compiler, operating system, and the underlying hardware architecture. The C standard only specifies *minimum* sizes and relationships. The exact sizes depend on how the compiler and system are implemented to best suit the target platform's characteristics (e.g., word size, memory organization). For example, an `int` might be 2 bytes on a 16-bit system, 4 bytes on a 32-bit system, and 8 bytes on a 64-bit system.